

Debugger



Índice

1. Introducción	2
2. Código a corregir	2
3. Desglose de errores	3
3.1 Error 1	3
3.2 Error 2	3
4. Eclipse	4
5. NetBeans	4
6. Visual Studio Code	6
7. Conclusión	8

1. Introducción

El uso de depuradores (*debuggers*) es una parte fundamental del desarrollo de software, ya que permite analizar el comportamiento de un programa durante su ejecución. A través del debugger es posible ejecutar el código paso a paso, inspeccionar el valor de las variables, detectar errores lógicos y comprender mejor el flujo del programa.

En esta práctica se utilizarán tres entornos de desarrollo integrados (IDE): **Eclipse**, **NetBeans** y **Visual Studio Code**, con el objetivo de aprender a utilizar sus herramientas de depuración y comparar su funcionamiento. Para ello, se empleará un programa previamente proporcionado, sobre el cual se aplicarán distintas técnicas de depuración.

2. Código a corregir

Nuestro profesor nos ha proporcionado el código que tenemos que corregir en la práctica. El código en específico es este:

```
package debugger;

import java.util.ArrayList;
import java.util.List;

public class Main {

    public static void main(String[] args) {
        List<Integer> numeros = new ArrayList<>();

        numeros.add(10);
        numeros.add(20);
        numeros.add(30);

        int resultado = calcularPromedio(numeros);

        System.out.println("Promedio: " + resultado);
    }

    public static int calcularPromedio(List<Integer> numeros) {
        int suma = 0;

        // ERROR 1: condición incorrecta del for
        for (int i = 0; i <= numeros.size(); i++) {
            suma = suma + numeros.get(i);
        }

        // ERROR 2: división entera (pierde decimales)
        int promedio = suma / numeros.size();

        return promedio;
    }
}
```

Como podemos observar los errores ya vienen determinados y no hace falta buscar otro error. Como se indica en esta práctica podemos comprobar que tenemos que utilizar el debugger en distintos IDEs.

3. Desglose de errores

A continuación vamos a corregir los dos errores que tenemos que reparar en el código

3.1 Error 1

El primer error es este:

```
// ERROR 1: condición incorrecta del for
for (int i = 0; i <= numeros.size(); i++) {
    suma = suma + numeros.get(i);
}
```

Como podemos comprobar este error no es sintáctico sino que es lógico y como nos lo indica el error se encuentra en el for. Este error básicamente cuando entramos a ese bucle no sale nunca debido a que la condición para que acabe es errónea. La solución es la siguiente:

```
// ERROR 1: condición incorrecta del for
for (int i = 0; i < numeros.size(); i++) {
    suma = suma + numeros.get(i);
}
```

Como podemos observar hemos quitado el igual porque si no nos aparecerá el error `java.lang.IndexOutOfBoundsException`

3.2 Error 2

El segundo error es este:

```
// ERROR 2: división entera (pierde decimales)
int promedio = suma / numeros.size();
return promedio;
```

Este como tal no es un error pero como el propio comentario lo dice este pierde decimales, tendremos que hacer un cambio de tipo de variable de entero a doble. Por tanto la solución es la siguiente:

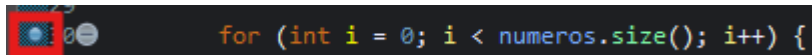
```
// ERROR 2: división entera (pierde decimales)
double promedio = suma / numeros.size();
return promedio;
```

Con todo esto tendríamos los errores cubiertos. A continuación veremos cómo utilizar el debugger en distintos IDEs

4. Eclipse

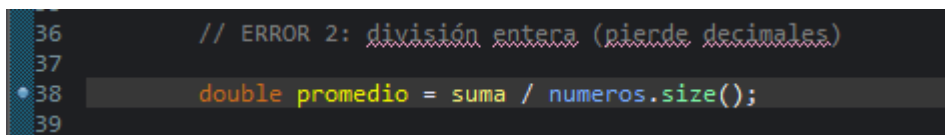
Crearemos un proyecto en Eclipse y crearemos un Main en donde pegaremos el código. A continuación añadiremos los breakpoints. Un breakpoint (o punto de interrupción) es una marca que se coloca en una línea concreta del código para indicar al programa que se detenga automáticamente ahí cuando se está ejecutando en modo depuración (debug).

Para colocarlo deberemos de hacer doble click en el número de la línea que queremos que se pare y se pondría. Sabremos que hemos puesto un breakpoint con este icono:



Una vez que hayamos puesto los breakpoints podremos ejecutar el programa. Una vez que se ejecute se parará donde hayamos puesto el breakpoint. Esto sucederá tanto en eclipse como en otros IDEs. Si no lo ejecutamos nos saldrá el error de lógica indicando que nuestro bucle for no está del todo correcto y que necesita cambios para que funcione de forma correcta. En nuestro caso como lo hemos corregido no nos lanzará ningún tipo de error

El siguiente breakpoint lo he puesto en siguiente punto:



Aquí como he dicho anteriormente no hay ningún tipo de error simplemente que al ser un double no perderemos ningún decimal y se incluyen todos los decimales saque salgan de la suma. Aquí como tal no haría falta que utilizemos el debugger.

Tras pararse en ese punto ya no habrá ninguna parada para el debugger y la aplicación se ejecutará sin parones a partir de ese punto. En el caso de que haya algún tipo de error el programa nos seguirá lanzando el error. Con todo esto sabremos cómo utilizar el debugger de eclipse.

5. NetBeans

Para NetBeans deberemos de crear también un proyecto como cualquier otro y pegaremos el código corregido dentro de la aplicación que vayamos a crear.:

```

package com.mycompany.programa;
import java.util.ArrayList;

import java.util.List;

public class Programa {

    public static void main(String[] args) {

        List<Integer> numeros = new ArrayList<>();

        numeros.add(10);

        numeros.add(20);

        numeros.add(30);

        double resultado = calcularPromedio(numeros);

        System.out.println("Promedio: " + resultado);

    }

    public static double calcularPromedio(List<Integer> numeros) {

        int suma = 0;

        // ERROR 1: condición incorrecta del for

        for (int i = 0; i < numeros.size(); i++) {

            suma = suma + numeros.get(i);

        }

        // ERROR 2: división entera (pierde decimales)

        double promedio = suma / numeros.size();

        return promedio;

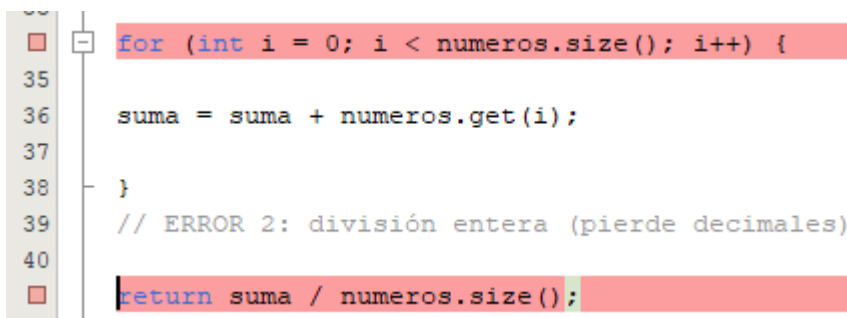
    }
}

```

Para abrir el debugger deberemos de hacer clic en el siguiente icono:



Para hacer los breakpoints deberemos de hacer lo mismo que en eclipse doble clic en el número de línea y cuando ejecutemos nuestro programa se parará en los breakpoints que queramos. Los breakpoints los pondremos en las siguientes líneas:



Una vez que entremos en el debugger para iniciarlo deberemos de hacer clic en el siguiente icono:



El primer icono es para empezar el programa como tal, el segundo icono es para parar completamente el programa y para volver al principio del programa, este corta completamente todo, el siguiente es parecido pero ese para el programa en el punto donde lo hayas dejado ejecutando, el siguiente básicamente es para reanudar la ejecución del programa por donde lo hayamos dejado.

Con todo esto tendríamos el debugger de NetBeans realizado y sabremos ejecutarlo en caso de que lo necesitemos utilizar.

6. Visual Studio Code

Para Visual Studio Code deberemos de crear un proyecto en Java y pegaremos el código indicado:

```

5  import java.util.ArrayList;
6
7  import java.util.List;
8
9  public class Programa {
10
11      Run | Debug
12      public static void main(String[] args) {
13          List<Integer> numeros = new ArrayList<>();
14
15          numeros.add(e: 10);
16
17          numeros.add(e: 20);
18
19          numeros.add(e: 30);
20
21          double resultado = calcularPromedio(numeros);
22
23          System.out.println("Promedio: " + resultado);
24      }
25
26      public static double calcularPromedio(List<Integer> numeros) {
27
28          int suma = 0;
29
30          // ERROR 1: condición incorrecta del for
31          for (int i = 0; i < numeros.size(); i++) {
32
33              suma = suma + numeros.get(i);
34
35          }
36          // ERROR 2: división entera (pierde decimales)
37          return suma / numeros.size();
38      }
39
40      }
41
42      }
43
44
45

```

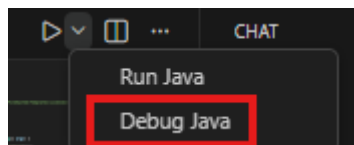
Como podemos observar el código está corregido. Aquí en Visual Studio para añadir un breakpoint es parecido pero esta vez para añadirlo nos aparecerá un punto rojo al lado del número para añadir dicho breakpoint. Añadiremos los siguientes breakpoint:

```

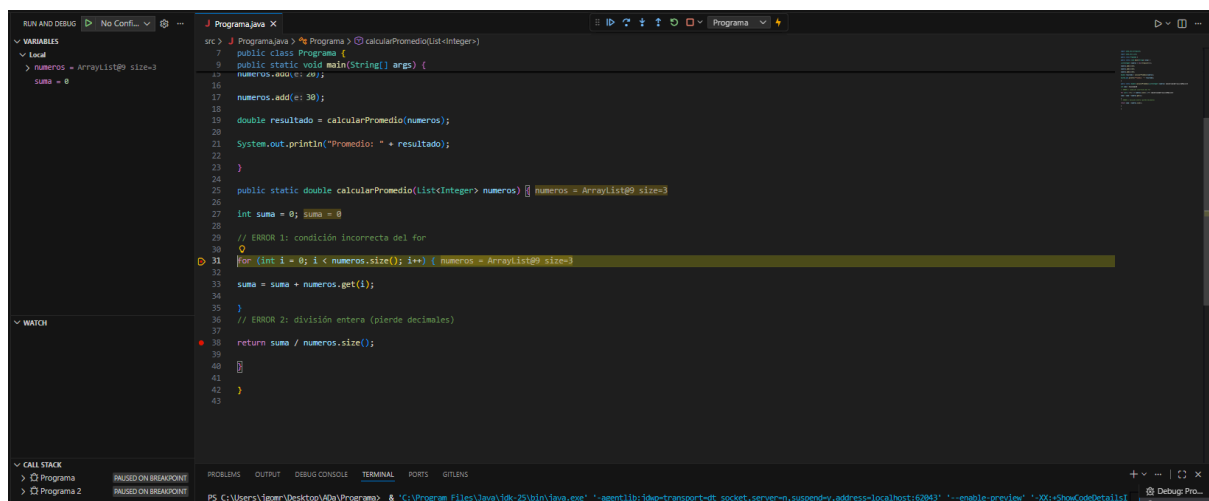
33 for (int i = 0; i < numeros.size(); i++) {
34
35     suma = suma + numeros.get(i);
36
37 }
38 // ERROR 2: división entera (pierde decimales)
39
40 return suma / numeros.size();

```

Como podemos observar cambia un poco pero la manera en la que funciona y se pone es de la misma manera que en los otros dos IDE. Para iniciar el debugger deberemos presionar el siguiente icono:



Deberemos de presionar la pestaña que nos aparece en el lateral donde se ejecuta nuestro programa. Y nos aparecerá la opción de debugger.



La pestaña de debugger de Visual Studio a mi parecer es algo más completa y más ordenada. Podemos observar en la esquina inferior izquierda los breakpoints que tenemos, en el lateral izquierdo podemos encontrar las variables y sus valores en esos momentos.

En la parte superior de la pantalla encontraremos un pequeño panel en el que podremos ejecutar el programa a nuestro antojo, podemos movernos hasta un breakpoint, retroceder, retroceder, etc. Podemos hacer infinidad de cosas con este debugger.

7. Conclusión

El uso del debugger y de los breakpoints resulta fundamental en el proceso de desarrollo de software, ya que permite analizar el funcionamiento interno de un programa durante su ejecución. Gracias a estas herramientas es posible detectar errores lógicos, comprender mejor el flujo del código y verificar el valor de las variables en cada momento.

A lo largo de la práctica se ha comprobado que, aunque Eclipse, NetBeans y Visual Studio Code presentan diferencias en su interfaz y forma de uso, todos ofrecen las funcionalidades básicas necesarias para realizar una depuración eficaz. El uso de breakpoints, la ejecución paso a paso y la inspección de variables son conceptos comunes en todos los entornos.

En conclusión, la depuración es una habilidad esencial para cualquier programador, y el dominio de estas herramientas facilita la detección de errores y mejora la calidad y fiabilidad del software desarrollado.